

¶ case file 4 of 7 · applied — filed 2026-02 · last verified 2026-07
status: shipped — web app live; the hosted classifier runs layer
1 only
you opened this file · jul 26, 2026

Applied

A job tracker that reads the search out of the inbox. Connect Gmail and Applied fetches your mail, names each message, and turns the noise into a pipeline of real applications you can act on. It shipped twice: first as a native macOS app, now as a hosted web app — and the two share one backend package.

role	designer and sole engineer — desktop app, web app, and classifier
timeframe	2026-02 to present
stack	next.js 16 · typescript · postgresql · python · setfit · onnx runtime
repo	yadava5/applied @ 36a2f54 · integration/web-migration (https://github.com/yadava5/applied/tree/36a2f54)
live demo	getapplied.vercel.app ↗ (https://getapplied.vercel.app)
system card	/system-card ↗ (https://getapplied.vercel.app/system-card)

private-safe proof: no email content — applied reads a real inbox, so this case file shows none of it. every receipt below terminates in source, a migration, a committed eval artifact, or a test run — never in a screenshot of mail. the

repository's own readme and docs/web_architecture.md still describe apps/web as an unwired scaffold and are deliberately not cited here: they are behind the code, and a stale doc is not evidence.

[problem] · § as found

The status of a job search scatters across Gmail, employer systems, and one-off messages. A spreadsheet can't keep up: updates get missed, rows get retyped, and the record drifts from the truth. The first answer was a desktop app, which meant the record only existed on one machine.

constraints —

ask for the least gmail access that can work — read-only, and metadata rather than message bodies.

classify noisy inbox messages into useful application states.

make row isolation the database's job, not the query writer's.

fit the whole classifier into a serverless slot, or be honest about which layers didn't fit.

never let a routine sync delete an application the current scan happened to miss.



fig. 1 – the sorting line – the 96-message eval set through rules → e5 similarity → gated setfit.

description: A drawn figure that runs – not a screenshot. Geometry from the repository's real 3-layer classifier path (rules → e5 similarity → gated SetFit); the four lanes and their counts are the committed eval set's own mix (96 messages, 12 per label × 8 labels); macro-F1 0.9791 from the committed baseline.

[architecture] · § fig. 2, inked

Gmail hands over metadata, the classifier names it, Postgres files it under an identity the database itself checks, and a Next.js dashboard reads it back. One backend package serves this and the desktop app; the desktop branch is where the two heavier classifier layers still live.

```
gmail
gmail.readonly — nothing wider
```

↓ read-only

```
metadata fetch
subject, from, date, snippet — no bodies
```

↓ subject + snippet

```
classifier
rules on the hosted path; e5 + setfit on the desktop one
```

↓ verdicts, scoped by user

```
postgres
rls on, force'd, per-transaction jwt claims
```

↓ pipeline state

```
next.js
pipeline board, review queue, stat tiles
```

off the spine —

classifier → swiftui layers 2–3 — desktop only

↗ no identity bound, no rows — the guc is unset
and rls denies

fig. 2 – the pipeline, inked. clay marks the gate: where a check can stop the run.

[decisions] · § as filed

d1 – fetch gmail metadata, never message bodies · accepted

The subject line, the sender, and Gmail’s own snippet are enough to name an application email.¹

¹ tradeoff – a body-blind classifier gives up signal on ambiguous mail, and buys a privacy boundary that holds without being trusted.

d2 – run the hosted classifier on the rules layer alone · accepted

torch, sentence-transformers, and SetFit do not fit a serverless function slot – not the size limit, and not the cold start.²

² tradeoff – the hosted verdict is weaker than the desktop one. the alternative was pretending otherwise, so the limit is written into the code, the tests, and the boundary rows below.

d3 – enforce row isolation in postgres, not in the handlers · accepted

Application-level `WHERE user\_id = ...` is one forgotten clause away from a leak.³

³ tradeoff – every transaction pays a `set\_config` round trip, and the app must run as a role that cannot bypass its own policies.

d4 – make routine sync additive; keep destruction behind a button
· accepted

A bounded scan of a large inbox will miss applications it already
found, and re-syncing wiped them.⁴

⁴ tradeoff – stale rows survive until an explicit rebuild, which
is the cheaper failure.

[validation] · § the receipts

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

at a glance – 11 of 11 terminate in pinned artifacts

method slip – eval protocol

sample: 96 messages – classifier_eval_v3.jsonl, 12 per label × 8
labels, balanced by design

mix: 65 core-positive · 17 edge-noise · 8 historical-miss · 6
core-negative scenarios

judge: predicted vs expected label → macro-F1; the ci gate fails
below the configured floor

run: 2026-03-03 – deterministic hybrid profile, committed as
baseline_hybrid_v3.json

repro: python -m jobtracker.scripts.evaluate_classifier --mode
hybrid --dataset data/evaluation/classifier_eval_v3.jsonl --
hybrid-profile deterministic

validation

walk the 11 claims →

claim

method · date

artifact ·
visibility

01 Connecting Gmail asks for gmail.readonly and nothing wider – and the consent step deliberately does not merge previously granted scopes.

read the oauth router and the settings field it draws its scope list from, at the pinned commit
2026-07-26

applied @ 36a2f54 · cloud/gmail_oauth.py (https://github.com/yadava5/applied/blob/36a2f54/backend/jobtracker/cloud/gmail_oauth.py)
applied @ 36a2f54 · tests/test_gmail_oauth_cloud.py (https://github.com/yadava5/applied/blob/36a2f54/backend/tests/test_gmail_oauth_cloud.py)
● [public]

02 The classifier is three layers – rules, e5 embeddings, SetFit – and SetFit stays off until its training gates are met.

ml strategy doc, read against the backend source
2026-07-26

applied @ 36a2f54 · docs/ML_STRATEGY.md (https://github.com/yadava5/applied/blob/36a2f54/docs/ML_STRATEGY.md)
● [public]

03 The hosted fetch reads metadata only – Subject, From, Date, and Gmail’s own snippet. Full message bodies are never downloaded on the web path.

read the cloud gmail client: messages.list plus batched messages.get(format=4/backend/jobtracker/cloud/read-only, no mutation
2026-07-26

applied @ 36a2f54 · cloud/gmail_client.py (https://github.com/yadava5/applied/blob/36a2f54/backend/jobtracker/cloud/gmail_client.py)
● [public]

04 I ran the backend suite locally at the pinned commit: 271 tests passed, 10 skipped, under the test/null-keyring environment.

local run against the public test tree; the 10 skips are the postgres rls module, which needs a live database
2026-07-26

applied @ 36a2f54 · backend/tests (<https://github.com/yadava5/applied/tree/36a2f54/backend/tests>)
● [public]

05 Rules and deterministic hybrid v3 gates both passed on 96 samples with macro-F1 0.9791.

committed
baseline,
deterministic
profile –
protocol in the
method slip
2026-03-03

applied @ 36a2f54 ·
baseline_hybrid_v3.
json (https://github.com/yadava5/applied/blob/36a2f54/backend/data/evaluation/baseline_hybrid_v3.json)
backend-ci run,
2026-04-20 ↗ (<https://github.com/yadava5/applied/actions/runs/2466506132>)
● [public]

06 The macOS Debug target built locally with xcodebuild against the JobTracker scheme, and the desktop app is still in the repository.

local build – no
build artifact
is published;
the source tree
is public and
linked
date not
recorded

applied @ 36a2f54 ·
apps/macos (<https://github.com/yadava5/applied/tree/36a2f54/apps/macos>)
⊖ [local – verified
on request]

07 The dashboard renders real applications: it reads the summary and application endpoints server-side and draws a pipeline board, a stage funnel, and a review queue from what comes back.

read the page
component and
the router it
calls, at the
pinned commit
2026-07-26

applied @ 36a2f54 ·
apps/web/app/(app)/
dashboard (<https://github.com/yadava5/applied/tree/36a2f54/apps/web/app/%28app%29/dashboard>)
applied @ 36a2f54 ·
cloud/applications.
py (<https://github.com/yadava5/applied/blob/36a2f54/backend/jobtracker/cloud/applications.py>)
● [public]

08 Row isolation is enforced by Postgres, not by the handlers: the app runs as a role that cannot bypass RLS, every transaction sets `request.jwt.claims` locally, and `user_credentials` is FORCE'd so even the table owner is held to the policies.

read the engine's begin-listener and the three rls migrations at the pinned commit
2026-07-26

applied @ 36a2f54 · database/connection.py (<https://github.com/yadava5/applied/blob/36a2f54/backend/jobtracker/database/connection.py>)
applied @ 36a2f54 · c5_force_user_credentials_rls.py (https://github.com/yadava5/applied/blob/36a2f54/backend/alembic/versions/c5_force_user_credentials_rls.py)
applied @ 36a2f54 · tests/test_rls_postgres.py (https://github.com/yadava5/applied/blob/36a2f54/backend/tests/test_rls_postgres.py)
● [public]

09 The hosted classifier runs the rules layer alone. On the serverless path it returns after layer 1 even when the rules were unsure – embeddings and SetFit are never imported there.

read the cloud short-circuit in the hybrid classifier and the import-hygiene test that holds it
2026-07-26

applied @ 36a2f54 · classifier/hybrid.py (<https://github.com/yadava5/applied/blob/36a2f54/backend/jobtracker/classifier/hybrid.py>)
applied @ 36a2f54 · tests/test_main_cloud.py (https://github.com/yadava5/applied/blob/36a2f54/backend/tests/test_main_cloud.py)
● [public]

outcomes

claim

method · date

artifact · visibility

10 Job updates land in a trackable pipeline instead of a spreadsheet – and now in a browser instead of on one Mac.

the product's own workflow, described – not a usage metric
2026-07-26

getapplied.vercel.app ↗ (<https://getapplied.vercel.app>)
● [public]

11 All three classifier layers do run in a browser – as a public Hugging Face Space, on an int8 ONNX export of the same model. That is a separate deployment from the web app, not the verdict
getapplied.vercel.app returns.

ported the local classifier and exported it quantized; both the space and the export script are inspectable
2026-07-26

huggingface.co/spaces/yadava5/jobtracker-classifier ↗ (<https://huggingface.co/spaces/yadava5/jobtracker-classifier>)
applied @ 36a2f54 · [ml/browser/export_onnx.py](https://github.com/yadava5/applied/blob/36a2f54/ml/browser/export_onnx.py) (https://github.com/yadava5/applied/blob/36a2f54/ml/browser/export_onnx.py)
● [public]

rows marked [local – verified on request] were run by me on personal or demo data · ci rows link the public run · repo pins are the exact commits verified 2026-07.

what i'm not claiming –

i'm not claiming the hosted app runs the full three-layer classifier. on vercel it runs the rules layer only – deliberately, because the model stack does not fit the function slot. embeddings and setfit stay on the desktop path and in the hugging face space.

i'm not claiming ci proves the rls policies enforce. the postgres rls suite skips unless a live database url is supplied, and no workflow supplies one; what is linked is the migrations, the per-transaction identity wiring, and the tests themselves.

i'm not citing the repository's readme or docs/web_architecture.md as evidence for the web app. both still describe apps/web as an unwired scaffold with a placeholder dashboard – they are behind the code, and this file cites the code.

no production email-volume or user numbers are claimed. source, migrations, and test runs are shown publicly; private email and application records are not shown.

[corrections] · § the register

erratum · 2026-07-26

Until today this file described a native macOS app whose “web beta is a scaffold”, and pinned every receipt at 3225eb4. That stopped being true: Applied ships as a hosted web app at getapplied.vercel.app. The caveat is retired because it is false now, not because it was wrong then – and the receipts are re-pinned to 36a2f54, the public head of the branch that carries the web app. The scaffold claim’s own successor is receipt 07.

erratum · 2026-07-26

The backend-suite row read 182 tests, audited at the old pin. Re-run at 36a2f54 on 2026-07-26 it is 271 passed and 10 skipped. Nothing was retracted – the tree grew, and the number moved with it. The 10 skips are named in the row rather than folded into the total.

note · 2026-07-26

The repository was renamed [yadava5/jobtracker](https://github.com/yadava5/jobtracker) → [yadava5/applied](https://github.com/yadava5/applied). GitHub still redirects the old links, so nothing this file ever published is broken; the pins name the current repository instead of relying on a redirect. The case-file route stays </projects/jobtracker/> for the same reason.

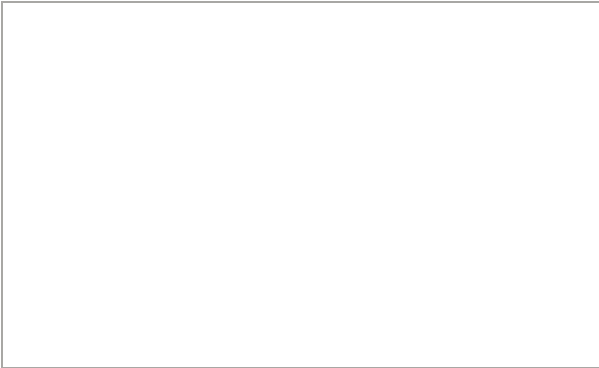


fig. 3 – local classification architecture.

description: rendered from the public repository structure · 2026-06 · open to inspect →

artifact index –

readme – the desktop-era record	↗ (https://github.com/yadava5/applied/blob/36a2f54/readme.md)
[repo] · yadava5/applied @ 36a2f54	
architecture docs	↗ (https://github.com/yadava5/applied/blob/36a2f54/docs/architecture.md)
[repo] · yadava5/applied @ 36a2f54	
ml strategy and evaluation gates	↗ (https://github.com/yadava5/applied/blob/36a2f54/docs/ml_strategy.md)
[benchmark] · yadava5/applied @ 36a2f54	
backend test suite	↗ (https://github.com/yadava5/applied/tree/36a2f54/backend/tests)
[repo] · yadava5/applied @ 36a2f54	
web app source	↗ (https://github.com/yadava5/applied/tree/36a2f54/apps/web)
[repo] · yadava5/applied @ 36a2f54	

next file — case file 5 of 7 · cadence — filed 2023-09 →

© 2026 ayush yadav — set
by hand

set in fraunces, newsreader &
fragment mono

two inks. one line. seven chapters.